

Functions in C: Implementations

Mohammad Maksood Akhter, PhD
Assistant Professor

Department of Computer Science & Engineering

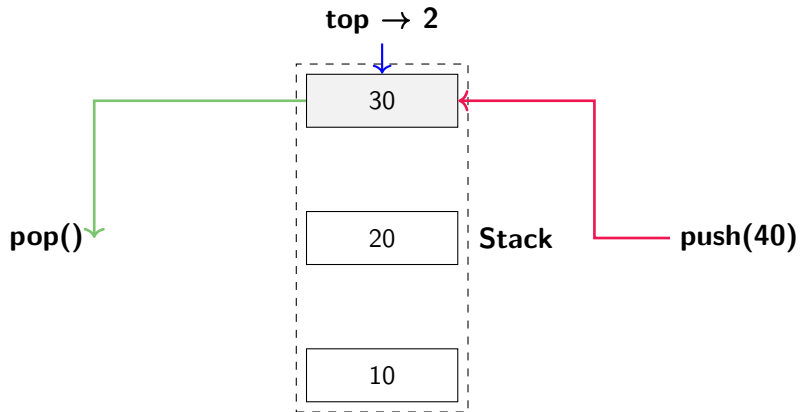
October 24, 2025



Note

A **stack** is a linear data structure that follows the **LIFO (Last In, First Out)** principle. The last element pushed is the first one popped.

- Can be implemented using **arrays** or **linked lists**.
- Operations:
 - **push()**: Adds an element to the top of the stack.
 - **pop()**: Removes the top element.
 - **top**: Pointer or index that tracks the last inserted element.
- $\text{top} = -1$ indicates an empty stack.



Tip

Push increases top by 1 before insertion.

Pop decreases top by 1 after removal.

```
1 #include <stdio.h>
2 #define MAX 10
3
4 int stack[MAX];
5 int top = -1;
6
7 // Function prototypes
8 void push(int value);
9 int pop();
10
11 void push(int value) {
12     if (top == MAX - 1)
13         printf("Stack Overflow!\n");
14     else {
15         top++;
16         stack[top] = value;
17     }
18 }
```

Listing 1: stack.c (Part 1)

```
1 int pop() {
2     if (top == -1)
3         printf("Stack Underflow!\n");
4     else
5         return stack[top--];
6 }
7
8 int main() {
9     push(10);
10    push(20);
11    printf("Popped: %d\n", pop());
12    return 0;
13 }
```

Listing 2: stack.c (Part 2)

Important

- Stack overflow occurs when pushing to a full stack.
- Stack underflow occurs when popping from an empty stack.
- Array-based stacks have fixed size; use linked lists for dynamic memory.
- **Recursion in C** internally uses a *function call stack*.

Tip

Stacks are heavily used in:

- Expression evaluation
- Reversal of data
- Function calls (activation records)

Note

An **Armstrong number** is a number that is equal to the sum of its own digits raised to the power of the number of digits.

- Example (3-digit): $153 = 1^3 + 5^3 + 3^3$
- Example (4-digit): $1634 = 1^4 + 6^4 + 3^4 + 4^4$
- To find whether a number is Armstrong:
 - ① Extract each digit using mod 10.
 - ② Push the digits into a stack.
 - ③ Count digits to get the power.
 - ④ Pop each digit, raise to the power, and sum the results.
 - ⑤ Compare the result with the original number.

```
1 #include <stdio.h>
2 #include <math.h>
3 #define MAX 10
4 int top = -1;
5 int stack[MAX];
6 void push(int value) {
7     top++;
8     stack[top] = value;
9 }
10 int pop() {
11     int value = stack[top];
12     top--;
13     return value;
14 }
15 int findarmstrong(int numb) {
16     int remainder, temp, count = 0,
```

Listing 3: armstrong.c (Part 1)

```
1 // Extract digits and push to stack
2 value = 0, j;
3 temp = numb;
4 while (numb > 0) {
5     remainder = numb % 10;
6     push(remainder);
7     count++;
8     numb /= 10;
9 }
10 numb = temp;
11 // Compute Armstrong sum
12 while (top >= 0) {
13     j = pop();
14     value += pow(j, count);
15 }
16 return (value == numb);
17 }
```

Listing 4: armstrong.c (Part 2)

```
1 void main() {  
2     int n;  
3     printf("Enter a number: ");  
4     scanf("%d", &n);  
5  
6     if (findarmstrong(n))  
7         printf("%d is an Armstrong number\n", n);  
8     else  
9         printf("%d is not an Armstrong number\n", n);  
0 }
```

Listing 5: main function snippet

Example

Input: 153 **Output:** 153 is an Armstrong number **Explanation:** $1^3 + 5^3 + 3^3 = 153$

- **Stack usage:** to temporarily store digits.
- **Function decomposition:** separate `findarmstrong()` from `main()`.
- **Mathematical logic:** uses `pow()` to raise digits to the correct power.
- **Reusability:** `findarmstrong()` can be used anywhere without rewriting the logic.

Tip

Try modifying the function to handle Armstrong numbers of any length or detect multiple Armstrong numbers within a range.

Concept (step-wise explanation)

- 1 Inspect the array and initialize both `max` and `min` to the first element.
- 2 Iterate through the array once (single pass).
- 3 For each element, compare with current `max` and `min` and update accordingly.
- 4 Return results via pointers (pass-by-reference) so the function can provide multiple outputs.

Code (Part A: function + helper)

```
1  #include <stdio.h>
2
3  void findMaxMin(int arr[], int n, int *max, int *min) {
4      *max = *min = arr[0];
5      for (int i = 1; i < n; i++) {
6          if (arr[i] > *max)
7              *max = arr[i];
8          if (arr[i] < *min)
9              *min = arr[i];
10     }
11 }
12 // end of Part A
```

Listing 6: maxmin.c - Part A

Code (Part B: main / usage)

```
1  int main() {  
2      int data[] = {12, 45, 23, 9, 56, 17};  
3      int max, min;  
4      findMaxMin(data, 6, &max, &min);  
5      printf("Maximum = %d, Minimum = %d\n", max, min);  
6      return 0;  
7  }  
8  // end of Part B
```

Listing 7: maxmin.c - Part B



Output:

Maximum = 56, Minimum = 9

Takeaway

- Use a single pass ($O(n)$) to compute both values efficiently.
- Prefer pass-by-reference over globals for cleaner APIs.
- Suitable pattern when a function needs to return several results.

Concept (step-wise explanation)

- 1 For two numbers a and b , if b is 0 then a is the GCD (base case).
- 2 Otherwise replace (a,b) with $(b, a \% b)$ and recurse — each step reduces the problem size.
- 3 Recursion unwinds when the base case is reached, producing the GCD.

Code (Part A: gcd function)

```
1  #include <stdio.h>
2
3  int gcd(int a, int b) {
4      if (b == 0)
5          return a;          // base case
6      else
7          return gcd(b, a % b); // recursive step
8  }
9  // end of Part A
```

Listing 8: gcdRecursive.c - Part A

Code (Part B: main / demonstration)

```
1  int main() {  
2      int x = 36, y = 60;  
3      printf("GCD of %d and %d is %d\n", x, y, gcd(x, y));  
4      return 0;  
5  }  
6  // end of Part B
```

Listing 9: gcdRecursive.c - Part B



Output:

GCD of 36 and 60 is 12

Takeaway

- Always define a clear base case to prevent infinite recursion.
- Recursive solutions are elegant for problems reducible to smaller subproblems.
- Be mindful of stack depth for very large inputs; consider iterative variants when needed.



Concept (step-wise explanation)

- 1 Read the binary number (as digits) and convert it to decimal by processing each bit with increasing power of two.
- 2 Convert the obtained decimal number to hexadecimal by repeated division by 16 and collecting remainders.
- 3 Present both steps as separate functions and compose them in `main` for clarity (functional decomposition).

Code (Part A: binary-to-decimal)

```
1      #include <stdio.h>
2      #include <math.h>
3
4      // Function to convert binary to decimal
5      int binToDec(long long bin) {
6          int dec = 0, i = 0, rem;
7          while (bin != 0) {
8              rem = bin % 10;
9              bin /= 10;
10             dec += rem * pow(2, i++);
11         }
12         return dec;
13     }
14     // end of Part A
```

Listing 10: binToHex.c - Part A

Code (Part B1: decimal-to-hex function)

```
1  #include <stdio.h>          // Function to convert decimal to hexadecimal
2  void decToHex(int dec) {
3      char hex[20];
4      int i = 0, temp;
5      if (dec == 0) {
6          printf("Hexadecimal: 0\n");
7          return; }
8      while (dec != 0) {
9          temp = dec % 16;
10         hex[i++] = (temp < 10) ? (temp + '0') : (temp + 'A' - 10);
11         dec /= 16;
12     }
13     printf("Hexadecimal: ");
14     for (int j = i - 1; j >= 0; j--)
15         printf("%c", hex[j]);
16     printf("\n");
17 }
```

// end of Part B1

Listing 11: binToHex.c - Part B1

Code (Part B2: main function)

```
1  int main() {  
2      long long binary;  
3      printf("Enter a binary number: ");  
4      scanf("%lld", &binary);  
5  
6      int decimal = binToDec(binary); // Part A  
7      decToHex(decimal);              // Part B  
8      return 0;  
9  }  
0  // end of Part B2
```

Listing 12: binToHex.c - Part B2



Example Run:

Input: 101101 Hexadecimal: 2D

Takeaway

- Break complex conversions into small functions for readability.
- Handle edge cases (e.g., input 0) explicitly.
- Functional decomposition supports testing and reuse.

Concept (step-wise explanation)

- 1 Preserve the original number for final comparison.
- 2 Reverse the digits of the number by repeatedly taking remainders and building the reversed integer.
- 3 Compare reversed number with the original; equality implies palindrome.

Code (Part A: isPalindrome function)

```
1  #include <stdio.h>
2
3  int isPalindrome(int num) {
4      int reversed = 0, temp = num, remainder;
5      while (temp != 0) {
6          remainder = temp % 10;
7          reversed = reversed * 10 + remainder;
8          temp /= 10;
9      }
10     return (num == reversed);
11 }
12 // end of Part A
```

Listing 13: palindrome.c - Part A

Code (Part B: main / usage)

```
1  int main() {  
2      int n;  
3      printf("Enter a number: ");  
4      scanf("%d", &n);  
5  
6      if (isPalindrome(n))  
7          printf("%d is a palindrome.\n", n);  
8      else  
9          printf("%d is not a palindrome.\n", n);  
10  
11     return 0;  
12 }  
13 // end of Part B
```

Listing 14: palindrome.c - Part B

Output Example:

Enter a number: 12321 12321 is a palindrome.

Takeaway

- Reversing digits is a simple and efficient check ($O(d)$ where d is digits).
- Be careful with signed numbers and overflow for very large inputs.
- Encapsulate logic in a function for reuse.



Concept (step-wise explanation)

- 1 Fix a position l and swap each character from l to r into that position.
- 2 Recursively permute the remaining substring $l+1 \dots r$.
- 3 After recursion, backtrack (swap back) to restore the string and try the next character.
- 4 When $l == r$, print the current permutation.

Code (Part A: swap + permute function)

```
1  #include <stdio.h>
2  #include <string.h>
3  void swap(char *x, char *y) {
4      char temp = *x;
5      *x = *y;
6      *y = temp;}
7  void permute(char *str, int l, int r) {
8      if (l == r)
9          printf("%s\n", str);
10     else {
11         for (int i = l; i <= r; i++) {
12             swap(&str[l], &str[i]);
13             permute(str, l + 1, r);
14             swap(&str[l], &str[i]); // backtrack
15         }
16     }
17 }
```

Listing 15: permute.c - Part A

Code (Part B: main / driver)

```
1  int main() {  
2      char word[20];  
3      printf("Enter a word: ");  
4      scanf("%s", word);  
5      int n = strlen(word);  
6      printf("All permutations are:\n");  
7      permute(word, 0, n - 1);  
8      return 0;  
9  }  
0  // end of Part B
```

Listing 16: permute.c - Part B



Example Output

Enter a word: ABC All permutations are: ABC ACB BAC BCA CBA CAB

Takeaway

- Recursion + backtracking enumerates all permutations ($n!$ results).
- Useful for combinatorial search problems; beware factorial growth.
- Use swap/backtrack idiom to avoid extra memory for string copies.

Thank You

Questions?